

华为最新笔试题及其分析

1. printf 的输出问题

```
printf("%d",total);//this is right  
printf(total);//this is wrong  
printf("hello");//but this is right
```

2. 整数类型的长度

char 1 个子节, 8 位

```
unsigned short [int]  
[signed] short int  
short 2 个字节, 16 位
```

```
[signed] int  
unsigned int  
int 型在 vc 里是 4 个子节, 32 位, 也可能是 16 位, 2 个字节
```

```
long [int]  
unsigned long [int]
```

long 型都是 32 位, 4 个字节

float 32 , 4

double 64, 8

long double 128, 16

char 8, 一个字节, 存放的实际上是字符的 ascii 码

3、找出错误并改正

```
char *my_cpy(char* src, int len){
char dest[1024];
memcpy(dest, src, len);
return dest;
}
```

上面的函数是否有问题, 如果有指出其所在, 如果没有, 给出函数功能描述。

答案:

1. 数组应该初始化
 2. memcpy 不判断是否越界, 所以调用前应该判断是否越界
 3. 不应该返回 rest, 因为这个数组是在函数内部申请的, 所以函数结束之后就会消失, 指针也会变成“野指针”, 所以指向非法地址
- 最后一个比较隐蔽

```
char *memcpy( char *dest, const char *src, int len )
{
char* pDest = (char*)dest;
char* pSrc = (char*)src;
int pos;
for(pos=0;pos<len;pos++)
{
pDest[pos] = pSrc[pos];
}
return (char*)pDest;
}
```

存在地问题就是没有判断指针是否非法 `assert(dest !=NULL || src != NULL);` 条件为 FLASE 显示

不调用其他函数，写一个 memcpy 的函数，函数原型为

```
void *memcpy(void *dest, void *src, size_t length);
```

-----利用好断言-----

```
/* memcpy —— 拷贝不重叠的内存块 */
void memcpy(void* pvTo, void* pvFrom, size_t size)
{
    void* pbTo = (byte*)pvTo;
    void* pbFrom = (byte*)pvFrom;
    ASSERT(pvTo != NULL && pvFrom != NULL);
    /* 内存块重叠吗？如果重叠，就使用 memmove */
    ASSERT(pbTo >= pbFrom + size || pbFrom >= pbTo + size);
    while(size-- > 0)
        *pbTo++ == *pbFrom++;
    return(pvTo);
}
```

常见函数编程：

```
char *strcpy(char *strDest, const char *strSrc)
{
    ASSERT(strDest != NULL && strSrc != NULL);
    char *addr = strDest;
    while(*strDest++ = *strSrc++)
        NULL; //NULL 可以省略，但更有利于编译器发现错误
    return addr;
}
```

```
void *memcpy(void *dest, const void *src, int count)
{
    ASSERT(dest != NULL && src != NULL);
    for(int i=0; i < count; i++)
    {
        dest = src;
    }
}
```

致力于打造中国最大、最专业和最具人性化的就业辅导和职场分享的互动平台。

```

}
}
int strcmp(const char*str1, const char *str2)
{
while (str1 != NULL && str2 != NULL)
{
if(*str1 < *str2) return -1;
else if(*str1 > *str2) return 1;
else { str1++; str2++;}
}
if(str1 == NULL && str2 != NULL)
return -1;
else if(str1 != NULL && str2 == NULL)
return 1;
else return 0;
}

//way2: more compact
int strcmp(const char*str1, const char *str2)
{
int i = strlen( str1 );
int j;
for(j=0; j<=i; j++)
{
if(str1[j] > str2[j]) return 1; //if str2 terminates, then str2[j]=0, str1[j]>str2[j], return
1;
else if(str1[j] < str2[j]) return -1;
else if(str1[j] == '') return 0;
}
}

//way3: optimize again.
int strcmp(const char * str1, const char * str2 )
{
while(1)
{
if(*str1 > *str2) return 1;
else if(*str1 < *str2) return -1;
else if(*str1 == '') return 0;
str1++;str2++;
}
}
}

```